



Instituto Tecnológico Superior de Lerdo

DESARROLLO DE APLICACIONES PARA DISPOSITIVOS MÓVILES UNIDAD 4

Portafolio Final

Nombre del Alumno(s)

Jesús Antonio Arellano Zapata

Número de Control

202310117

Docente:

Jesús Salas Marín

Fecha de Entrega

12/06/2026

Índice

Introducción	3
Desarrollo	4
Proyecto 2: Análisis de Datos con Dart	4
Proyecto 3: Primera Aplicación en Flutter	6
Proyecto 4: Creación de un Mini Reproductor de Música en Flutter	9
Reflexión final	16

https://github.com/SoyTonion/PortafolioMoviles_JesusAntonioArellanoZapata.git

Introducción

El propósito de este portafolio es recopilar y presentar las prácticas y proyectos desarrollados durante el semestre en la materia de Desarrollo de Aplicaciones para Dispositivos Móviles. A través de estas actividades se trabajaron temas fundamentales del lenguaje Dart y del framework Flutter, aplicando conceptos básicos y avanzados para la creación de programas de consola y aplicaciones móviles.

Durante el desarrollo del portafolio se fortalecieron competencias relacionadas con la programación estructurada, programación orientada a objetos, manejo de funciones, uso de clases, herencia, manipulación de datos, lectura y escritura de archivos JSON, creación de interfaces gráficas, navegación entre pantallas y manejo de estado en aplicaciones móviles. Cada proyecto permitió aplicar conocimientos teóricos en ejercicios prácticos, ayudando a comprender mejor la lógica de programación y la estructura de una aplicación.

Desarrollo

Proyecto 2: Análisis de Datos con Dart

Objetivo del proyecto:

Construir una aplicación de consola en Dart que cargue y analice datos almacenados en formato JSON.

Problema planteado:

Se necesitaba leer datos desde un archivo JSON, procesarlos mediante funciones, realizar búsquedas y filtros, calcular estadísticas básicas y exportar un resumen en un nuevo archivo JSON.

Tecnologías utilizadas:

Dart, Visual Studio Code, JSON

Conceptos aplicados:

Lectura de archivos, escritura de archivos, clases, objetos, listas, funciones, estructuras de control, Null Safety, búsqueda, filtrado y estadísticas básicas.

Explicación del funcionamiento:

El programa carga registros desde un archivo JSON y los convierte en objetos de una clase Registro. Después permite mostrar todos los datos, buscar por nombre, filtrar por edad o salario, calcular estadísticas como salario promedio, edad mínima, edad máxima y cantidad de registros. Finalmente, permite exportar un resumen en formato JSON.

Capturas del proyecto:

```
main.dart x
main.dart > ...
1 import 'dart:io';
2 import 'dart:convert';
3
4 class Registro {
5   String _nombre;
6   int _edad;
7   double _salario;
8
9   Registro(this._nombre, this._edad, this._salario);
10
11   String get nombre => _nombre;
12   int get edad => _edad;
13   double get salario => _salario;
14
15   factory Registro.fromJson(Map<String, dynamic> json) {
16     return Registro(
17       json['nombre'] ?? 'Sin nombre',
18       json['edad'] ?? 0,
19       (json['salario'] ?? 0).toDouble(),
20     );
21   }
22
23   Map<String, dynamic> toJson() {
24     return {
25       'nombre': _nombre,
26       'edad': _edad,
27       'salario': _salario,
28     };
29   }
30 }

datos.json x
datos.json > ...
1 [
2   {
3     "nombre": "Ana",
4     "edad": 22,
5     "salario": 8500.0
6   },
7   {
8     "nombre": "Carlos",
9     "edad": 30,
10    "salario": 12000.0
11  },
12  {
13    "nombre": "María",
14    "edad": 27,
15    "salario": 10500.0
16  },
17  {
18    "nombre": "Luis",
19    "edad": 35,
20    "salario": 15000.0
21  },
22  {
23    "nombre": "Sofía",
24    "edad": 24,
25    "salario": 9800.0
26  }
27 ]

PS C:\Users\tonyz\Documents\Uni\Dispositivos Moviles\Proyecto Unidad 2 (Dart)> dart run main.dart
Datos cargados correctamente.

===== MENÚ DE ANÁLISIS DE DATOS =====
1. Mostrar todos los registros
2. Buscar por nombre
3. Filtrar por edad mínima
4. Filtrar por salario mínimo
5. Mostrar estadísticas
6. Exportar resumen a JSON
7. Salir
Seleccione una opción: 5
Cantidad de registros: 5
Salario promedio: $11160.00
Edad mínima: 22
Edad máxima: 35
```

Dificultades encontradas:

Una dificultad fue manejar correctamente la lectura del archivo JSON y evitar errores si el archivo no existía o tenía datos inválidos.

Soluciones aplicadas:

Se utilizó manejo de errores con try-catch, validación de existencia del archivo y operadores de Null Safety para evitar fallos durante la carga de datos.

Proyecto 3: Primera Aplicación en Flutter

Objetivo del proyecto:

Crear una primera aplicación en Flutter que genere pares de palabras aleatorias, permita guardar favoritos y navegar entre pantallas.

Problema planteado:

Se buscaba desarrollar una aplicación móvil básica pero interactiva, aplicando widgets, estado, botones, diseño visual, navegación y listas.

Tecnologías utilizadas:

Flutter, Dart, Visual Studio Code.

Conceptos aplicados:

Widgets, StatelessWidget, StatefulWidget, manejo de estado, ChangeNotifier, Provider, botones, tarjetas, NavigationRail, LayoutBuilder, ListView y ListTile.

Explicación del funcionamiento:

La aplicación muestra un par de palabras aleatorio en una tarjeta. El usuario puede presionar el botón Next para generar una nueva palabra o el botón Like para guardarla como favorita. También se agregó un riel de navegación con las secciones Home y Favorites. En la página de favoritos se muestran las palabras guardadas en una lista. Además, se utilizó LayoutBuilder para hacer que el riel de navegación se adapte al tamaño de la pantalla.

Capturas del proyecto:

```
class MyHomePage extends StatelessWidget {
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();
    var pair = appState.current;

    return Scaffold(
      body: Column(
        children: [
          Text('Una idea GENIAL:'),
          Text(pair.asLowerCase),
        ],
      ),
    );
  }
}

class BigCard extends StatelessWidget {
  const BigCard({
    super.key,
    required this.pair,
  });

  final WordPair pair;

  @override
  Widget build(BuildContext context) {
    return Text(pair.asLowerCase);
  }
}
```

```
Text('Una idea GENIAL:'),
BigCard(pair: pair),
```

```

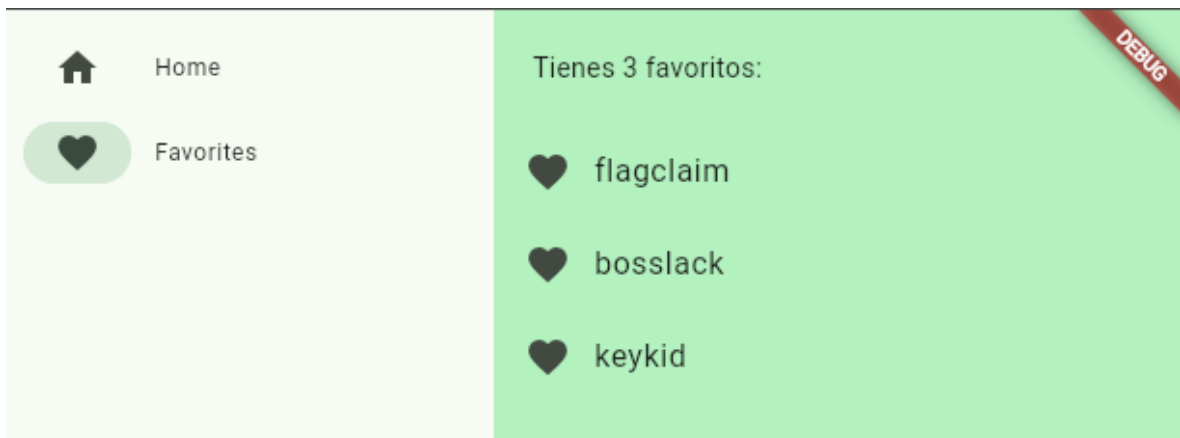
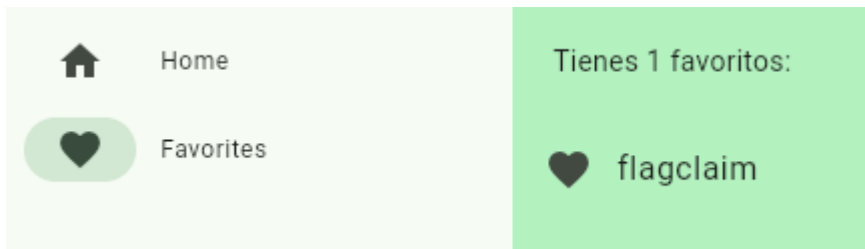
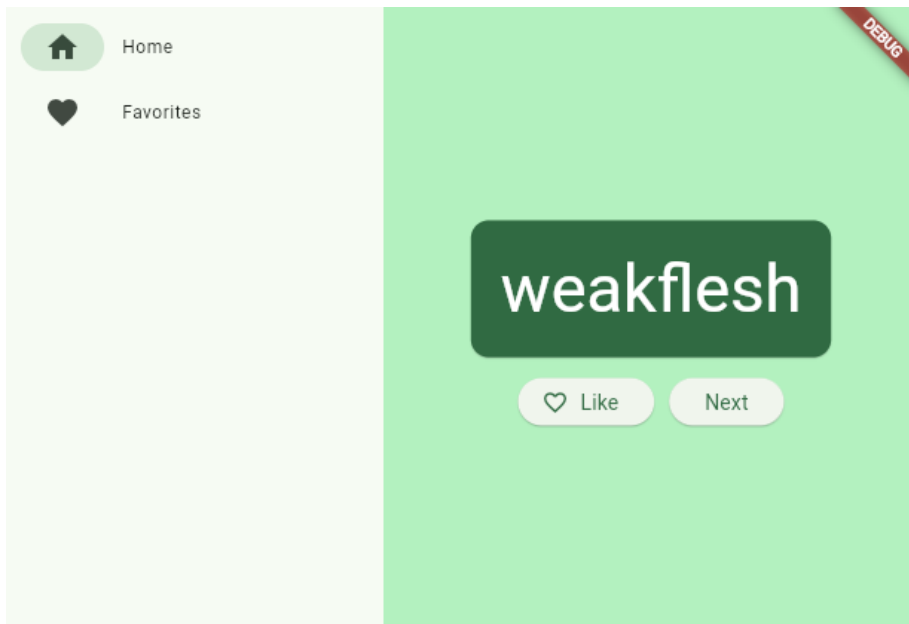
class _MyHomePageState extends State<MyHomePage> {
  var selectedIndex = 0;
  ⚡
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Row(
        children: [
          SafeArea(
            child: NavigationRail(
              extended: false,
              destinations: [
                NavigationRailDestination(
                  icon: Icon(Icons.home),
                  label: Text('Home'),
                ), // NavigationRailDestination

```

```

161
162 class FavoritesPage extends StatelessWidget {
163   @override
164   Widget build(BuildContext context) {
165     var appState = context.watch<MyAppState>();
166
167     if (appState.favorites.isEmpty) {
168       return Center(
169         child: Text('No hay favoritos todavía.'),
170       ); // Center
171     }
172
173     return ListView(
174       children: [

```



Dificultades encontradas:

Una dificultad fue comprender cómo separar la aplicación en varios widgets y cómo cambiar entre pantallas usando el índice seleccionado.

Soluciones aplicadas:

Se dividió el código en widgets como BigCard, GeneratorPage y FavoritesPage. También se convirtió MyHomePage en un StatefulWidget para controlar la navegación mediante selectedIndex.

Proyecto 4: Creación de un Mini Reproductor de Música en Flutter

Objetivo del proyecto

El objetivo de este proyecto fue crear una aplicación básica en Flutter que funcionara como un mini reproductor de música. La aplicación debía mostrar una interfaz visual con una imagen de portada, el nombre de la canción, una barra de progreso y botones de control. Además, debía reproducir un archivo de audio real utilizando la librería `just_audio`.

Problema planteado

El problema planteado consistió en desarrollar una aplicación que no solo simulara visualmente un reproductor de música, sino que también pudiera reproducir audio de forma funcional. Para esto fue necesario configurar archivos de audio e imagen dentro del proyecto, agregar dependencias externas, controlar la reproducción y actualizar la barra de progreso conforme avanzaba la canción.

Tecnologías utilizadas

- Flutter, Dart, Visual Studio Code

Conceptos aplicados

En este proyecto se aplicaron conceptos importantes del desarrollo móvil con Flutter, como el uso de `StatefulWidget`, manejo de estado, configuración de dependencias en `pubspec.yaml`, uso de archivos locales mediante `assets`, reproducción de audio, `streams`, barra de progreso con `Slider`, widgets personalizados y organización del proyecto en varios archivos.

También se utilizaron widgets como `Scaffold`, `AppBar`, `Column`, `Row`, `Container`, `Text`, `IconButton`, `Slider` y `StreamBuilder`, los cuales permitieron estructurar y actualizar la interfaz del reproductor.

Explicación del funcionamiento

La aplicación inicia desde el archivo `main.dart`, donde se ejecuta la clase principal `MusicPlayerApp`. Esta clase carga la pantalla `MusicPlayerScreen`, que contiene toda la interfaz y lógica del reproductor.

Dentro de la pantalla principal se declara un objeto `AudioPlayer`, encargado de cargar y reproducir la canción almacenada en la carpeta `assets`. Al iniciar la pantalla, se ejecuta el método que carga el archivo de audio. Después, el botón principal permite reproducir o pausar la canción dependiendo del estado actual.

La barra de progreso se construye mediante un widget personalizado llamado `SeekBar`, el cual recibe la duración total, la posición actual y la posición almacenada en búfer. Estos datos se obtienen mediante streams y se combinan para actualizar la interfaz mientras la canción se reproduce.

La interfaz muestra una imagen circular del artista, el nombre de la canción, la barra de progreso y tres botones: canción anterior, reproducir/pausar y canción siguiente. Aunque los botones de anterior y siguiente están colocados visualmente, la función principal desarrollada fue la reproducción y pausa de la canción actual.

Capturas del proyecto

> android	3
▼ assets	4
🔊 baby.mp3	5
🖼️ justin.jpg	6
> ios	7
> lib	8
> lib	9

```
lib > main.dart > MusicPlayerApp > build
1  import 'package:flutter/material.dart';
2
Run | Debug | Profile
3  void main() {
4      runApp(const MusicPlayerApp());
5  }
6
7  class MusicPlayerApp extends StatelessWidget {
8      const MusicPlayerApp({super.key});
9
10     @override
11     Widget build(BuildContext context) {
12         return MaterialApp(
13             debugShowCheckedModeBanner: false,
14             title: 'Music Player',
15
16             home: const MusicPlayerScreen(),
17         ); // MaterialApp
18     }
19 }
```

```
lib > position_data.dart > PositionData > PositionData
1  class PositionData {
2      final Duration position;
3      final Duration bufferedPosition;
4      final Duration duration;
5
6      PositionData(this.position, this.bufferedPosition, this.duration);
7  }
```

```
lib > seek_bar.dart > SeekBar > build
1  import 'package:flutter/material.dart';
2
3  class SeekBar extends StatelessWidget {
4      final Duration duration;
5      final Duration position;
6      final Duration bufferedPosition;
7      final ValueChanged<Duration>? onChangeEnd;
8
9      SeekBar({
10         required this.duration,
11         required this.position,
12         required this.bufferedPosition,
13         this.onChangeEnd
14     });
15
```

```
lib > music_player_screen.dart > MusicPlayerScreen > MusicPlayerScreen
1  import 'package:flutter/material.dart';
2  import 'position_data.dart';
3  import 'seek_bar.dart';
4  import 'package:just_audio/just_audio.dart';
5  import 'package:rxdart/rxdart.dart';
6
7
8  class MusicPlayerScreen extends StatefulWidget {
9      const MusicPlayerScreen({super.key});
10
```

```
15
16  class _MusicPlayerScreenState extends State<MusicPlayerScreen> {
17      late AudioPlayer _audioPlayer;
18      bool isPlaying = false;
19      String currentTrack = 'Baby';
20      String artist = 'Justin Bieber';
21
22      @override
23      void initState(){
24          super.initState();
25          _audioPlayer = AudioPlayer();
26          _loadAudio();
27      }
28
29      Future<void> _loadAudio() async {
```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: ☐ const Color(0xFF2C2C54),
    appBar: AppBar(
      backgroundColor: ☐ const Color(0xFF2C2C54),
      elevation: 0,
      title: const Text(
        'Hear Me!',
        style: TextStyle(color: ☒ Colors.white),
      ), // Text
      centerTitle: true,
    ), // AppBar
  );
}

```

```

StreamBuilder<PositionData>(
  stream: _positionDataStream,
  builder: (context, snapshot) {
    final positionData = snapshot.data;
    return SeekBar(
      duration: positionData?.duration ?? Duration.zero,
      position: positionData?.position ?? Duration.zero,
      bufferedPosition: positionData?.bufferedPosition ?? Duration.zero,
      onChangeEnd: (newPosition) {
        _audioPlayer.seek(newPosition);
      }
    ); // SeekBar
  },
), // StreamBuilder

```

```

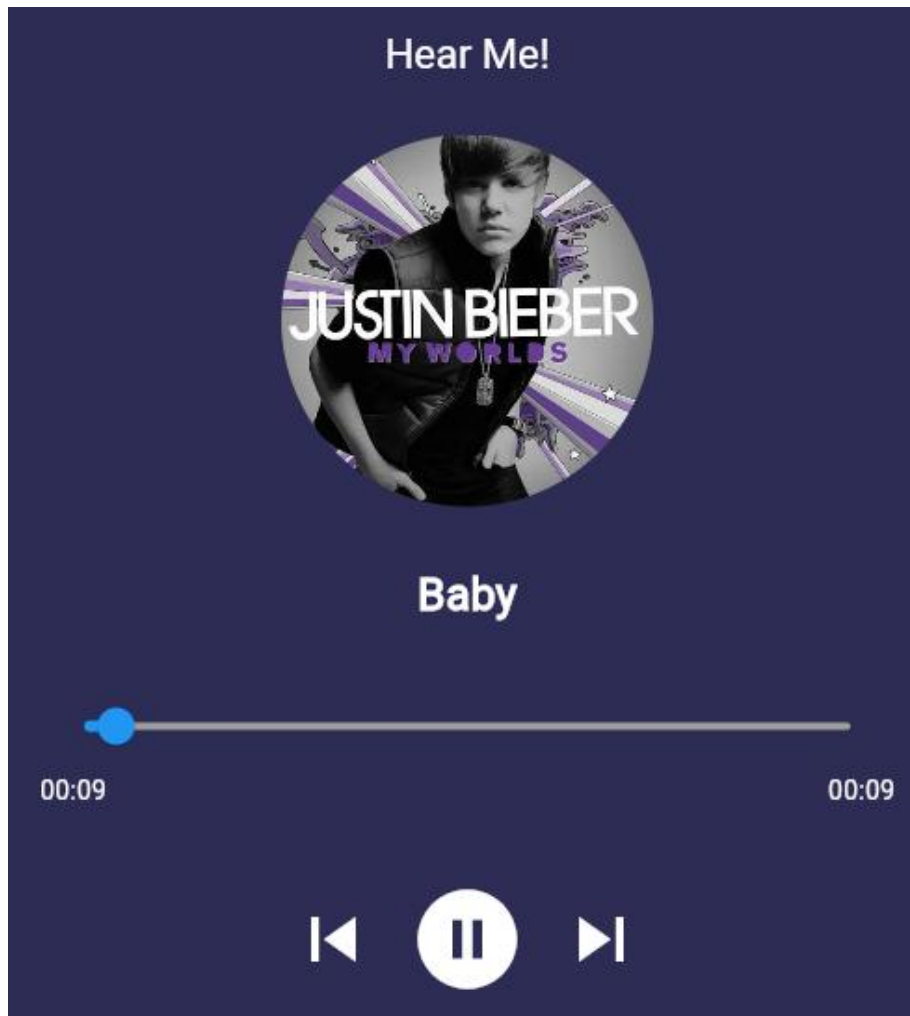
const SizedBox(height: 30,),
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    IconButton(
      iconSize: 48,
      icon: const Icon(
        Icons.skip_previous,
        color: Colors.white,
      ), // Icon
      onPressed: (){},
    ), // IconButton
    IconButton(
      iconSize: 64,
      icon: Icon(
        isPlaying
          ? Icons.pause_circle_filled
          : Icons.play_circle_fill,
        color: Colors.white,
      ), // Icon
      onPressed: _playPause,
    ), // IconButton
  ],
)

```

```

    IconButton(
      iconSize: 48,
      icon: const Icon(
        Icons.skip_next,
        color: Colors.white,
      ), // Icon
      onPressed: (){},
    ), // IconButton
  ],
) // Row
],
) // Column
), // Padding
); // Scaffold
}
}

```



Dificultades encontradas

Una de las principales dificultades fue configurar correctamente las dependencias del proyecto, ya que al usar librerías externas como `just_audio` y `rxdart` era necesario agregarlas correctamente en `pubspec.yaml` y ejecutar `flutter pub get`.

Otra dificultad fue la configuración de los assets, porque si las rutas del audio o de la imagen no coincidían con las declaradas en el código, la aplicación no podía cargar los archivos. También se presentaron errores de sintaxis relacionados con comas, paréntesis, imports y widgets personalizados.

Además, al ejecutar la aplicación en Windows apareció un problema relacionado con los symlinks, debido a que los plugins requieren soporte para enlaces simbólicos.

Soluciones aplicadas a las dificultades

Para solucionar los problemas con las dependencias, se revisó el archivo `pubspec.yaml`, se agregaron correctamente `just_audio` y `rxdart`, y se ejecutó el comando `flutter pub get` para descargar los paquetes necesarios.

Para resolver los errores con los assets, se verificó que los archivos estuvieran dentro de la carpeta correspondiente y que las rutas usadas en el código coincidieran exactamente con las rutas registradas en `pubspec.yaml`.

Los errores de sintaxis se corrigieron revisando la estructura de los archivos, agregando las comas faltantes, cerrando correctamente paréntesis y llaves, e importando los archivos necesarios como `position_data.dart` y `seek_bar.dart`.

Finalmente, para solucionar el problema de ejecución en Windows, se identificó que no era un error del código, sino una configuración del sistema. La solución fue activar el Modo de desarrollador de Windows o ejecutar la aplicación en un emulador o dispositivo compatible.

Reflexión final

El proyecto que considero más difícil fue la primera aplicación en Flutter, porque integró varios conceptos al mismo tiempo, como widgets, manejo de estado, navegación, diseño responsivo y listas dinámicas. A diferencia de los ejercicios de consola en Dart, en Flutter no solo se trabaja con la lógica del programa, sino también con la forma en que los elementos aparecen e interactúan dentro de la interfaz.

El proyecto más útil fue el de análisis de datos con Dart, ya que permitió trabajar con archivos JSON, búsqueda, filtros y estadísticas. Este tipo de proyecto se puede relacionar con situaciones reales donde se necesita cargar información, procesarla y generar reportes.

La habilidad que más mejoró fue la organización del código. A lo largo de las prácticas aprendí a dividir los programas en funciones, clases y widgets, evitando tener todo el código en una sola parte. También mejoré en el uso de programación orientada a objetos, especialmente con clases, constructores, encapsulación y herencia.

Sobre la asignatura aprendí que el desarrollo de aplicaciones móviles requiere combinar lógica de programación, estructura de datos, diseño visual y manejo de eventos. También comprendí que Dart es una base importante para trabajar con Flutter, ya que muchos conceptos aprendidos en consola se aplican después en aplicaciones móviles. En general, estas actividades me ayudaron a entender mejor cómo se construyen programas funcionales y cómo se puede avanzar desde ejercicios básicos hasta una aplicación interactiva.

CONCLUSIÓN

En conclusión, el desarrollo de estos proyectos me permitió reforzar mis conocimientos sobre Dart y Flutter, pasando de programas de consola a aplicaciones móviles con interfaz gráfica. A través del proyecto de Análisis de Datos con Dart aprendí a trabajar con archivos JSON, clases, funciones, listas, búsquedas, filtros y estadísticas básicas, comprendiendo cómo procesar información de forma organizada. Por otra parte, con la Primera Aplicación en Flutter aprendí a crear una interfaz interactiva, utilizar widgets, manejar el estado de la aplicación, agregar botones, guardar favoritos y navegar entre pantallas. En general, estas actividades me ayudaron a mejorar mi lógica de programación, mi organización del código y mi comprensión sobre cómo se construyen aplicaciones funcionales, combinando estructura, diseño e interacción con el usuario.